

**Student Meal Planner**

# **Risk Management**

*A Managing Document for the Grubs4Scrubs Project*

Date: April 2026

Author: Yao Cheng Zhou

## Why Risk Management for a Solo Project

Risk management sounds like something for big team projects, but solo projects have risks too. Actually, they have more in some ways. There's no teammate to catch your mistakes, no one to take over if you're stuck, and the deadline doesn't move because you're one person. I identified these risks during the project and documented how I dealt with (or plan to deal with) each one.

### Risk 1: No Entity Framework Allowed

**Risk:** The course requires raw ADO.NET for all database access. Most tutorials, Stack Overflow answers, and documentation examples assume EF Core. Finding resources and examples for the repository pattern with raw SQL was harder than expected.

**Impact:** High. Without EF Core, every query has to be written by hand. No migrations, no automatic mapping, no LINQ. Development is slower and there's more room for SQL errors.

**How I handled it:** I researched the repository pattern specifically with ADO.NET (documented in the Database & Data Access research doc). I created a private `MapRecipe()` helper method inside the repository to avoid repeating the SqlDataReader-to-object mapping logic. Parameterized queries everywhere to prevent SQL injection. It took longer than EF Core would have, but the code is clean and I understand every query.

### Risk 2: CSS Inconsistency Across Pages

**Risk:** The project started with a mix of Tailwind CSS and custom CSS. Different pages looked like they belonged to different apps. As more pages were added, the inconsistency got worse and debugging CSS became unpredictable.

**Impact:** Medium. The app still worked, but the visual inconsistency made it look unfinished and made styling changes risky (changing one thing could break another page's layout).

**How I handled it:** I removed Tailwind entirely and standardized on custom CSS with a strict `ComponentName-element-subelement` naming convention. Converted every page. Documented the decision in the Frontend Design Decisions research doc. The risk is now eliminated.

### Risk 3: Connection String and Database Setup Issues

**Risk:** Getting the backend to connect to SQL Server caused repeated errors. I started with LocalDB, then switched to full SQL Server via the mssql VS Code extension. The connection string had the wrong server format ( `localhost\\MSSQLLocalDB` instead of `localhost` ).

**Impact:** High. Without a working database connection, the API can't do anything. This blocked backend development for a while.

**How I handled it:** Updated the connection string in `appsettings.json` to point to `Server=localhost`. Made sure the Grubs4Scrubs database existed in the correct SQL Server instance. Created the Recipes table with the right schema. Tested the connection before moving on to building endpoints.

### Risk 4: Scope Creep

**Risk:** It's tempting to keep adding features. Dietary filters, recipe ratings, social sharing, multi-user households. Every idea sounds good until you realize the semester has a deadline.

**Impact:** Medium to high. If I try to build everything, nothing gets finished properly. The portfolio needs working features, not half-built ones.

**How I handled it:** I used MoSCoW prioritization from the project analysis. Must-haves are recipe browsing, meal planning, dashboard, and user auth. Should-haves are shopping list and Google OAuth. Everything else is a could-have or won't-have for this semester. When I get an idea for a new feature, I check it against the MoSCoW list before starting.

### Risk 5: Solo Developer Bottleneck

**Risk:** Everything depends on me. If I get stuck on a problem or make a bad architectural decision early, there's no team member to catch it or help. Being sick for a week means a week of zero progress.

**Impact:** Medium. It hasn't caused a major issue yet, but it's always a background risk.

**How I handled it:** Regular check-ins with my coach to catch problems early. Writing documentation as I go (so the reasoning behind decisions is captured even if I forget later). Using Git so I can always roll back to a working state. Breaking work into small chunks so getting stuck on one thing doesn't block everything else.

## **Risk 6: Namespace and Build Errors During Restructuring**

**Risk:** Splitting the backend into four separate projects introduced a wave of namespace errors, missing project references, and NuGet package issues. Files that worked in the flat project suddenly couldn't find their dependencies.

**Impact:** Medium. Didn't break anything permanently, but it blocked development for a couple of hours while I tracked down every broken reference.

**How I handled it:** Fixed each error one at a time. Updated namespaces to match new project names. Installed NuGet packages in the correct projects (Microsoft.Data.SqlClient in DataAccess, not API). Added project references where needed. Lesson learned: next time, commit after each step of a restructuring instead of doing everything at once.

## **Risk 7: Authentication System Complexity**

**Risk:** Building JWT authentication from scratch (not using a library like ASP.NET Identity) means handling password hashing, token generation, token validation middleware, and claims management manually. Each piece has small details that are easy to get wrong.

**Impact:** High. A broken auth system means either users can't log in, or worse, the system is insecure (unhashed passwords, tokens that don't validate, etc.).

**How I handled it:** I built it step by step and tested each piece before moving to the next. BCrypt for password hashing (industry standard, generates its own salt). JWT tokens with HMAC-SHA256 signing and proper claims. Token validation middleware configured in Program.cs with issuer, audience, and key checks. I ran into a lot of small bugs during implementation (wrong variable names, reversed if-checks, autocorrect changing property names), but catching them one at a time meant the final system is solid. The frontend isn't wired up yet, but the backend endpoints are testable through Swagger.

## **Risk 8: Time Split Between Two Projects**

**Risk:** Working on Grubs4Scrubs (solo) and Media Bazar (group) at the same time means neither project gets full attention. Some weeks one project dominates and the other stalls.

**Impact:** Medium. Both projects have deadlines, and falling behind on one creates pressure on the other. Documentation for both projects competes for the same time.

**How I handled it:** I tried to block specific days for each project instead of switching between them within the same day. Context-switching between two different codebases, two different tech stacks, and two different team structures is where the time disappears. It's not fully solved, but having dedicated blocks helps.

## Risk Summary

| Risk                                | Likelihood  | Impact | Status                                    |
|-------------------------------------|-------------|--------|-------------------------------------------|
| No EF Core allowed                  | Certain     | High   | Mitigated (ADO.NET patterns established)  |
| CSS inconsistency                   | Was certain | Medium | Eliminated (Tailwind removed)             |
| Database connection issues          | Happened    | High   | Resolved (connection string fixed)        |
| Scope creep                         | Ongoing     | High   | Managed (MoSCoW prioritization)           |
| Solo developer bottleneck           | Ongoing     | Medium | Managed (coach check-ins, Git, docs)      |
| Namespace errors from restructuring | Happened    | Medium | Resolved (references fixed)               |
| Auth system complexity              | Happened    | High   | Mitigated (built and tested step by step) |
| Time split between projects         | Ongoing     | Medium | Managed (day-blocking)                    |